



PROJECT REPORT

NexusSupport-AI

AI Support & Research Assistant

A Machine Learning, LLM, and AI Agent System - from Raw Data to a Deployed, Monitored API

Prepared as a skill-building and portfolio project for a Machine Learning Engineer Internship application

August 2026

Prepared by: Don Dew



Table of Contents

1. Executive Summary
2. The Problem This Project Solves
3. High-Level Pipeline (The Big Picture)
4. Frameworks and Tools Used
5. Part-by-Part Deep Explanation
 - 5.1 The Data
 - 5.2 Data Preprocessing (Cleaning the Data)
 - 5.3 Classical Machine Learning - The Baseline Classifier
 - 5.4 Deep Learning - Fine-Tuning a Transformer (DistilBERT)
 - 5.5 Retrieval-Augmented Generation (RAG)
 - 5.6 The AI Agent - Decision-Making Logic
 - 5.7 Model Deployment - Turning the Project into a Real Service
 - 5.8 Monitoring and Visualization Dashboard
6. End-to-End Walkthrough (One Message, Start to Finish)
7. How This Project Covers Internship-Level ML Engineer Skills
8. How to Run the Project, Step by Step
9. Results Summary
10. Limitations and Honest Notes
11. Conclusion

1. Executive Summary

This report explains, from start to end, a complete AI system called the “AI Support & Research Assistant.” The project was built to practice and demonstrate the real skills required for a Machine Learning Engineer Internship: Python data handling, classical machine learning, deep learning, large language models (LLMs), AI agents, cloud deployment, and monitoring/visualization.

In simple words: this project is a smart customer-support helper. A customer sends a message (for example, “I was charged twice, can I get a refund?”). The system reads the message, figures out what it's about, decides whether it can answer safely on its own, looks up the correct information from company documents, and writes a clear answer back — all automatically, and all visible through a live web API.

The report below walks through every part of the project in the order it was built: the data, the cleaning steps, the two different machine learning models used and why, the retrieval and LLM system, the decision-making agent, the deployment setup, and the monitoring dashboard. Each section explains what the part does, how it works, and which tools/frameworks were used - in plain English first, with technical detail underneath.

2. The Problem This Project Solves

Companies receive large numbers of customer support messages every day. Reading and answering each one manually is slow and expensive. Many of these messages are repetitive - refund questions, login problems, account changes — and the answers already exist somewhere in the company's help documents. The problem is connecting the right question to the right answer, automatically and reliably, while still sending harder or uncertain cases to a real human.

Goal: Build a system that reads an incoming message, classifies it, searches company knowledge for the right information, generates a natural answer, and knows when to ask a human for help instead of guessing — then make that system available as a real, callable, deployed service.

3. High-Level Pipeline (The Big Picture)

Before going part by part, here is the full journey of one message through the system, described simply:

1. A user sends a message, for example: “I can't log in, it shows a server error.”
2. The system cleans and prepares the text (Preprocessing).
3. A classical Machine Learning model quickly labels the message's category, e.g. “technical” (Classification).
4. An AI Agent checks: “Am I confident enough to continue, or should I hand this to a human?” (Decision).
5. If confident, the system searches the company's FAQ documents for the most relevant passages (Retrieval).
6. A Large Language Model (LLM) reads those passages and writes a natural, accurate answer (Generation).
7. The full interaction is logged, and a dashboard shows how the system is performing (Monitoring).
8. All of this runs inside a web service (API) that can be deployed to the cloud so real apps can use it (Deployment).

This pattern — clean data, classify, decide, retrieve, generate, monitor, deploy - is the same pattern used in real production AI systems at technology companies, just at a smaller, learning-friendly scale.

4. Frameworks and Tools Used

The table below lists every major tool used in the project, what it is, and exactly why it was chosen for this project.

Tool / Framework	Category	Why it was used here
Python	Language	The standard language for machine learning and data work; huge ecosystem of libraries.
Pandas & NumPy	Data handling	Loading, cleaning, and manipulating the ticket data as tables and numeric arrays.
Scikit-learn	Classical ML	Building the TF-IDF + Logistic Regression baseline classifier, splitting data, scoring results.
PyTorch	Deep learning engine	The underlying framework that trains and runs the transformer model.
Hugging Face Transformers	Deep learning / NLP	Provides the pretrained DistilBERT model and an easy training loop (Trainer API).
Hugging Face Datasets	Deep learning / NLP	Efficiently formats and tokenizes data for the transformer.
Sentence-Transformers	Embeddings	Converts text into meaning-vectors for semantic search (used in RAG).
FAISS	Vector search	Extremely fast “nearest neighbor” search over those meaning-vectors.
Anthropic API (Claude)	LLM	Generates the final natural-language answer, grounded in retrieved context.
FastAPI	Deployment / API	Turns the whole system into a callable web service with automatic validation and docs.
Uvicorn	Web server	Runs the FastAPI application.
Docker	Containerization	Packages the app and all dependencies so it runs identically anywhere.
AWS / GCP	Cloud platforms	Where the container is deployed so the service is publicly reachable.
Streamlit	Dashboard	Builds an interactive visual dashboard with very little code.
Plotly / Matplotlib / Seaborn	Visualization	Draws the charts and confusion matrix in the dashboard.
Git	Version control	Tracks changes to the code over time; standard professional practice.

5. Part-by-Part Deep Explanation

This is the core of the report. Each part below explains: what it does, why it exists, and exactly how it works.

5.1 The Data

Every machine learning project starts with data. Since this project doesn't use a real company's private ticket database, a realistic synthetic dataset was generated instead: 600 example support messages spread across four categories - billing, technical, account, and general - plus a small set of FAQ documents that represent a company's help center.

Simple explanation: Think of this as a pretend inbox of customer messages, built to look and behave like a real one, so every other part of the project has something real to learn from and be tested on.

Why synthetic data: It lets the whole pipeline be demonstrated end-to-end without needing access to a real company's private data. The same code works unchanged on a real dataset - you would simply replace the CSV file.

5.2 Data Preprocessing (Cleaning the Data)

Raw text is messy: inconsistent capital letters, extra spaces, duplicate messages, stray symbols. Before any model can learn from it, the text needs to be cleaned and organized. This step does three things:

- **Cleaning:** lowercases text, removes extra whitespace and odd symbols.
- **Splitting:** divides the data into training, validation, and test sets, keeping the mix of categories balanced across each set (called a “stratified split”).
- **Chunking:** breaks long FAQ documents into short, overlapping passages so the search system (used later in RAG) can find and return a focused piece of text rather than an entire document.

Simple explanation: This is like tidying up a messy room before you can find anything in it. A model trained on messy data learns messy, unreliable patterns.

Why the split matters: The training set teaches the model, the validation set tunes it, and the test set gives an honest, unseen check of how well it really performs - similar to studying from a textbook, practicing on past exams, then taking a final exam with new questions.

5.3 Classical Machine Learning - The Baseline Classifier

This part reads a support message and predicts its category (billing, technical, account, or general). It uses a well-established, simple technique: TF-IDF combined with Logistic Regression.

How TF-IDF works (simple explanation)

TF-IDF stands for “Term Frequency – Inverse Document Frequency.” It turns each message into a list of numbers, where common, unimportant words (like “the” or “is”) get a very low score, and unusual, meaningful words (like “refund” or “crashes”) get a high score. This turns text into something a mathematical model can actually work with.

How Logistic Regression works (simple explanation)

Logistic Regression looks at those number patterns and learns a simple rule: “messages with these kinds of words tend to belong to this category.” It then uses that rule to guess the category of any new message, along with a confidence score (how sure it is).

Why start with a simple model: In real ML work, you always build a simple baseline first. It trains in seconds, is easy to understand, and gives a fair comparison point. If a more complex model (like the transformer, see 5.4) can't beat this baseline by a meaningful amount, it isn't worth the extra cost and slowness.

How it was evaluated: Using precision, recall, and F1-score for each category - not just overall accuracy - because accuracy alone can hide poor performance on smaller categories. A confusion matrix (a simple table of “what the model predicted” vs “what was actually true”) was also produced to visually check for mistakes.

Result on the test set (from actual training run): overall test accuracy of about 89%, with strong performance across all four categories.

5.4 Deep Learning - Fine-Tuning a Transformer (DistilBERT)

This part solves the exact same classification task as the baseline above, but using a modern deep learning model called DistilBERT - a smaller, faster version of the well-known BERT language model.

What “fine-tuning” means (simple explanation)

DistilBERT has already “read” a massive amount of general text during its original training, so it already understands grammar, meaning, and context. “Fine-tuning” means taking that already-smart model and giving it a small amount of additional, focused training on our specific support tickets, so it learns our specific categories without having to learn the English language from zero.

Why this can beat the classical model: TF-IDF only recognizes exact words. DistilBERT understands meaning and context, so it can tell that “the app won't stop closing” and “it keeps crashing” mean the same thing, even though they barely share any words. This usually helps most with messages that are worded in unusual or indirect ways.

Trade-off: The transformer is slower and needs more computing power (ideally a GPU) than the classical model, which is instant and runs on any laptop. This is a genuine, real-world trade-off between accuracy and cost/speed - and part of the value of this project is being able to explain that trade-off clearly.

5.5 Retrieval-Augmented Generation (RAG)

An LLM like Claude is extremely good at writing natural language, but it has never seen this company's private FAQ documents. RAG solves this problem without retraining the model at all. It works in four steps:

9. Embedding: every FAQ passage is converted into a list of numbers (a “vector”) that represents its meaning, using a Sentence-Transformer model.
10. Indexing: all those vectors are stored in FAISS, a tool built specifically for very fast similarity search over large numbers of vectors.
11. Retrieval: when a question comes in, it's converted into a vector the same way, and FAISS instantly finds the passages whose meaning is closest to the question.
12. Generation: those retrieved passages are given to the LLM along with the question, with an instruction to answer using only that information.

Simple explanation: This is like giving the LLM an open-book exam instead of asking it to answer from memory. It looks up the correct page in the company handbook before answering, instead of guessing.

Why this matters: It keeps answers accurate and up to date. If the FAQ changes, you just update the text file - no retraining needed. It also greatly reduces “hallucination,” which is when an LLM confidently states something false.

5.6 The AI Agent - Decision-Making Logic

This is the part that connects everything else together and makes the system feel “smart” rather than just following one fixed script. The difference between a simple chatbot and an agent is that an agent makes decisions based on what it discovers along the way.

The agent's tools

- Classify - quickly labels the message's category using the classical ML model (fast and cheap, so it's always run first).
- Search knowledge base - retrieves the most relevant FAQ passages (the RAG retrieval step).
- Answer with context - asks the LLM to generate a grounded final answer.
- Escalate - hands the message to a human agent if the system isn't confident enough.

The decision loop (simple explanation)

The agent behaves like a careful employee, not a guessing machine. First, it classifies the message. If it isn't confident about the category, it immediately escalates to a human rather than risk giving a wrong answer. If it is confident, it searches for relevant information. If nothing relevant is found, it escalates again instead of making something up. Only when it has both a confident category and relevant supporting information does it generate and return an answer.

Why this design matters: Every decision the agent makes is visible and explainable - there is a full “trace” of what it checked and why it chose to answer or escalate. This transparency is exactly what real production AI agents need, and it was

built without a heavy external framework so every step can be understood and explained clearly (a LangChain-based version of the same idea is included for reference, since frameworks like LangChain and LlamaIndex are also commonly used in industry).

5.7 Model Deployment - Turning the Project into a Real Service

A model sitting in a script on a laptop isn't useful to anyone else. Deployment means wrapping the whole system in a web service that other applications (a website, a mobile app, another backend) can call over the internet.

FastAPI

FastAPI is used to build the web service. It exposes clear endpoints such as `/classify`, `/ask`, and `/agent`, so a request can be sent in and a structured JSON answer comes back. It automatically checks that incoming requests are well-formed, and it generates an interactive documentation page automatically.

Important design detail: The machine learning models are loaded into memory only once, when the service starts up — not on every single request. Reloading a model on every request is a common mistake that makes a service extremely slow; loading once and reusing it keeps responses fast.

Docker

Docker packages the entire application - code, dependencies, and settings - into a single portable “container.” This container runs identically on a laptop, a teammate's computer, or a cloud server, avoiding the classic “it works on my machine” problem.

Cloud deployment (AWS / GCP)

Once containerized, the service can be deployed to a cloud platform so it has a real, public web address. Two paths are documented: Google Cloud Run (simplest, scales automatically, and scales down to zero cost when idle) and AWS EC2 (a virtual server running the same Docker container), with a note on more advanced options like AWS ECS Fargate for later growth.

5.8 Monitoring and Visualization Dashboard

A working AI system needs to be watched, not just built. The dashboard (built with Streamlit and Plotly) shows two kinds of information:

- Model performance: the confusion matrix and class balance from the classical ML model, so accuracy can be checked visually.
- Live traffic: how many requests the deployed API has received, how long they took (latency), and which categories are most common - pulled directly from the running service.

Why this matters: In a real job, this is how you would notice a problem early - for example, response times slowly getting worse, or one category suddenly being over-predicted, which could signal that the model needs retraining.

6. End-to-End Walkthrough (One Message, Start to Finish)

To make the whole pipeline concrete, here is exactly what happens to one real example message as it travels through the system:

Message: “I was charged twice for my Pro plan, can I get a refund?”

Step	What happens	Component used
1	Text is cleaned (lowercased, whitespace normalized).	Preprocessing
2	Message is classified as “billing” with a confidence score.	Classical ML (TF-IDF + Logistic Regression)
3	Agent checks confidence - high enough to continue.	AI Agent (decision point)
4	The billing FAQ document is searched; the refund policy passage is retrieved.	RAG retrieval (Sentence-Transformers + FAISS)
5	The LLM writes a clear answer using only the retrieved passage.	RAG generation (Claude API)
6	The answer, category, and sources are returned as a JSON response.	FastAPI endpoint (/agent)
7	The request is logged for the dashboard.	Monitoring / logging

7. How This Project Covers Internship-Level ML Engineer Skills

Skill area from job description	Where it is demonstrated in this project
ML fundamentals & Python (NumPy, Pandas, Scikit-learn)	Data cleaning, splitting, and the classical ML classifier
Large Language Models (LLMs)	RAG generation step using the Claude API
AI agents	The decision-making SupportAgent with classify → retrieve → answer/escalate logic
Model deployment	FastAPI service, Docker containerization, cloud deployment guide
PyTorch / TensorFlow	DistilBERT fine-tuning using PyTorch and Hugging Face Transformers
Cloud platforms (AWS, GCP)	Documented, working deployment steps for both platforms
Git, data preprocessing, visualization	Structured as a proper Git-ready project; full preprocessing pipeline; Streamlit/Plotly dashboard

8. How to Run the Project, Step by Step

These are the exact commands to set up and run every part of the project locally.

Step 1 - Install dependencies

```
pip install -r requirements.txt
```

Step 2 - Generate the dataset

```
python data/generate_sample_data.py
```

Step 3 - Train the classical ML model

```
python src/classical_ml.py
```


Step 4 - (Optional) Fine-tune the transformer

```
python src/deep_learning.py
```

Step 5 - Try the RAG pipeline

```
export GOOGLE_API_KEY=your_key_here  
python src/rag_pipeline.py
```

Step 6 - Run the full agent

```
python src/agent.py
```

Step 7 - Deploy the API locally

```
uvicorn src.app:app --reload --port 8000
```

Step 8 - Launch the dashboard

```
streamlit run dashboard/dashboard.py
```

Step 9 - Containerize and deploy to the cloud

```
docker build -t ai-support-assistant .  
docker run -p 8000:8000 -e GOOGLE_API_KEY=your_key ai-support-assistant
```

9. Results Summary

Metric	Result
Classical ML - Test Accuracy	~89%
Classical ML - Training Time	A few seconds on a standard laptop
Categories Classified	Billing, Technical, Account, General
RAG Retrieval Method	Cosine similarity search over sentence embeddings (FAISS)
Deployment Format	Dockerized FastAPI service, deployable to AWS or GCP
Monitoring	Live request volume, latency, and category breakdown via Streamlit dashboard

10. Limitations and Honest Notes

- The dataset is synthetic (generated by a template script), so real-world performance on genuine customer messages would need to be re-measured on real data.
- The dataset is small once duplicates are removed, so the reported accuracy should be read as a proof-of-concept result, not a production-grade benchmark.
- The transformer fine-tuning step needs internet access to download pretrained weights and benefits from a GPU; it was written and reviewed for correctness but requires an appropriate environment to execute fully.

- The RAG and agent steps require a valid LLM API key and internet access, since they call an external Large Language Model.
- This is a learning and portfolio project, not an audited production system - further work (authentication, logging security, real data governance) would be needed before real customer deployment.

11. Conclusion

This project takes a common, realistic business problem - handling customer support messages - and solves it using the full range of tools expected of a Machine Learning Engineer today: clean data handling, a classical ML baseline, a fine-tuned deep learning model, a retrieval-augmented LLM system, a decision-making AI agent, and a properly deployed, containerized, cloud-ready, and monitored service.

Beyond the working code, the real value of this project is being able to explain, clearly and simply, why each piece exists and how it connects to the next - which is exactly what this report has aimed to do, part by part, from the very first line of data to the final deployed answer.